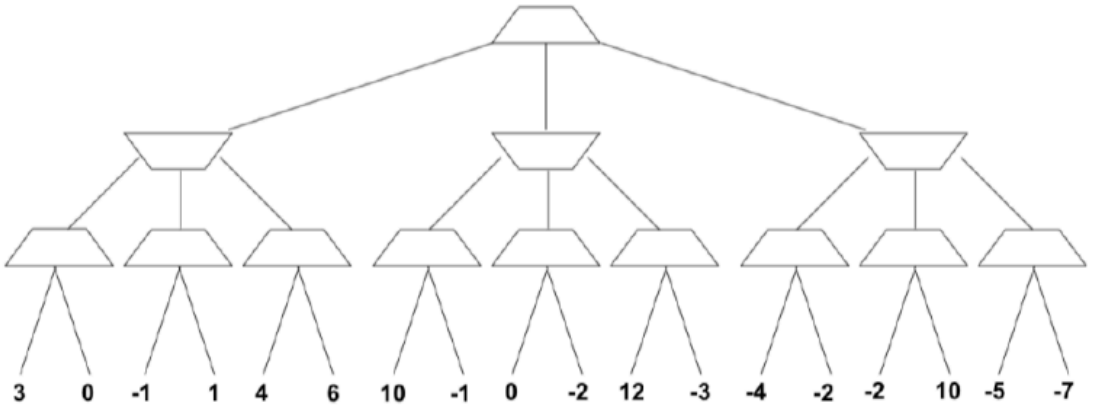
BÀI TẬP LÝ THUYẾT
TÌM KIẾM, CSP VÀ GAME TRONG AI (ĐÁP ÁN)

Họ và tên	Lê Đoàn Kim Ngân
MSSV / Lớp	3123410231 / DCT1236

Bài 1. Minimax và Alpha-Beta Pruning



Hình 1. Cây trò chơi dùng cho Minimax và Alpha-Beta (nguồn: rec3.pdf, trang 1)

Cây có gốc MAX, 3 nút MIN, mỗi nút MIN có 3 nút MAX, mỗi nút MAX có 2 lá (tổng 18 lá). Để dễ trình bày mình đặt tên: ba nút MIN là MIN1, MIN2, MIN3; chín nút MAX tầng dưới lần lượt là a, b, c (con của MIN1), d, e, f (con của MIN2), g, h, i (con của MIN3). Các cặp lá từ trái sang phải: a(3, 0), b(-1, 1), c(4, 6), d(10, -1), e(0, -2), f(12, -3), g(-4, -2), h(-2, 10), i(-5, -7).

a) Giá trị các nút trung gian (Minimax)

Tính từ dưới lên. Nút MAX lấy giá trị lớn nhất của các con, nút MIN lấy nhỏ nhất.

Tính 9 nút MAX ở tầng dưới (mỗi nút lấy lớn nhất của 2 lá):

- a = max(3, 0) = 3
- b = max(-1, 1) = 1
- c = max(4, 6) = 6
- d = max(10, -1) = 10
- e = max(0, -2) = 0
- f = max(12, -3) = 12
- g = max(-4, -2) = -2
- h = max(-2, 10) = 10
- i = max(-5, -7) = -5

Tính 3 nút MIN (mỗi nút lấy nhỏ nhất của 3 nút MAX con):

- MIN1 = min(a, b, c) = min(3, 1, 6) = 1
- MIN2 = min(d, e, f) = min(10, 0, 12) = 0
- MIN3 = min(g, h, i) = min(-2, 10, -5) = -5

Tính gốc: max(MIN1, MIN2, MIN3) = max(1, 0, -5) = 1.

b) Nhánh MAX nên chọn và giá trị trò chơi

Ba nhánh có giá trị 1, 0, -5 nên MAX chọn nhánh bên trái (MIN1). **Giá trị trò chơi là 1.**

c) Alpha-Beta từ trái sang phải

Khởi tạo ở gốc $\alpha = -\infty$, $\beta = +\infty$. Quy ước: nút MAX cập nhật α, nút MIN cập nhật β, cắt tỉa khi $\alpha \geq \beta$.

MIN1 ($\alpha=-\infty$, $\beta=+\infty$). Nút a $(-\infty, +\infty)$: lá 3 làm $\alpha=3$, lá 0 không đổi, trả về 3. MIN1 có $\beta=3$. Nút b $(-\infty, 3)$: lá -1 rồi lá 1 làm $\alpha=1$, vẫn nhỏ hơn $\beta=3$ nên không cắt, trả về 1. MIN1 có $\beta=1$. Nút c $(-\infty, 1)$: lá 4 làm $\alpha=4 \geq \beta=1$ nên **cắt lá 6**. MIN1 = min(3, 1, ≥4) = 1. Về gốc: $\alpha=1$.

MIN2 ($\alpha=1$, $\beta=+\infty$). Nút d $(1, +\infty)$: lá 10 làm $\alpha=10$, lá -1 không đổi, trả về 10, MIN2 có $\beta=10$. Nút e $(1, 10)$: lá 0 và lá -2 đều không vượt $\alpha=1$, trả về 0. MIN2 có $\beta=0$. Lúc này $\alpha=1 \geq \beta=0$ nên **cắt nút f** (hai lá 12 và -3). MIN2 trả về 0.

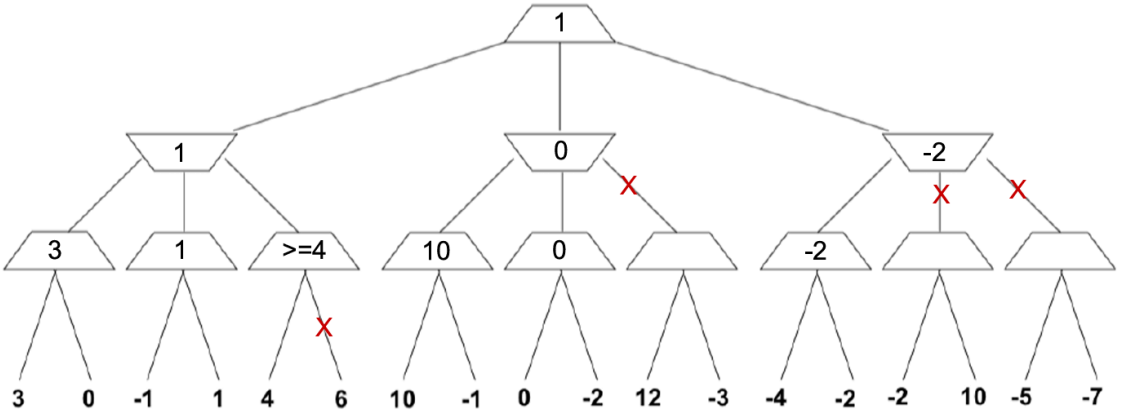
MIN3 ($\alpha=1$, $\beta=+\infty$). Nút g $(1, +\infty)$: lá -4 rồi lá -2, trả về -2. MIN3 có $\beta=-2 \leq \alpha=1$ nên **cắt hai nút h và i** (các lá -2, 10, -5, -7). MIN3 trả về -2 (chỉ là cận trên, nhưng đủ để biết nó không được chọn).

Gốc: max(1, 0, ≤-2) = 1, vẫn đúng như Minimax.

Bảng theo dõi Alpha-Beta (đi từ trên xuống, trái sang phải):

Nút	**[α, β] lúc vào**	**Diễn biến**	**Kết quả**
Gốc	$[-\infty, +\infty]$	Lần lượt duyệt MIN1, MIN2, MIN3; α tăng từ $-\infty$ lên 1 sau MIN1	1
MIN1	$[-\infty, +\infty]$	Duyệt a, b, c; β giảm $+\infty \rightarrow 3 \rightarrow 1$	1
a	$[-\infty, +\infty]$	Lá 3 làm $\alpha = 3$; lá 0 không đổi	3
b	$[-\infty, 3]$	Lá -1 rồi lá 1 làm $\alpha = 1$, vẫn nhỏ hơn $\beta = 3$ nên không cắt	1
c	$[-\infty, 1]$	Lá 4 làm $\alpha = 4 \geq \beta = 1$, cắt	≥ 4 (X lá 6)
MIN2	$[1, +\infty]$	Duyệt d, e; β giảm $+\infty \rightarrow 10 \rightarrow 0$, lúc này $\beta = 0 < \alpha = 1$ nên dừng	0 (X nút f)

Nút	[α, β] lúc vào	Diễn biến	Kết quả
d	[1, +∞]	Lá 10 làm α = 10; lá -1 không đổi	10
e	[1, 10]	Lá 0 và lá -2 đều không vượt α = 1	0
MIN3	[1, +∞]	Duyệt g; β = -2 < α = 1 nên dừng	≤ -2 (X nút h, i)
g	[1, +∞]	Lá -4 và lá -2	-2



Hình 1 (đã điền). Giá trị, [α, β] lúc vào nút và các nhánh bị cắt (X).

d) Các lá không cần thăm

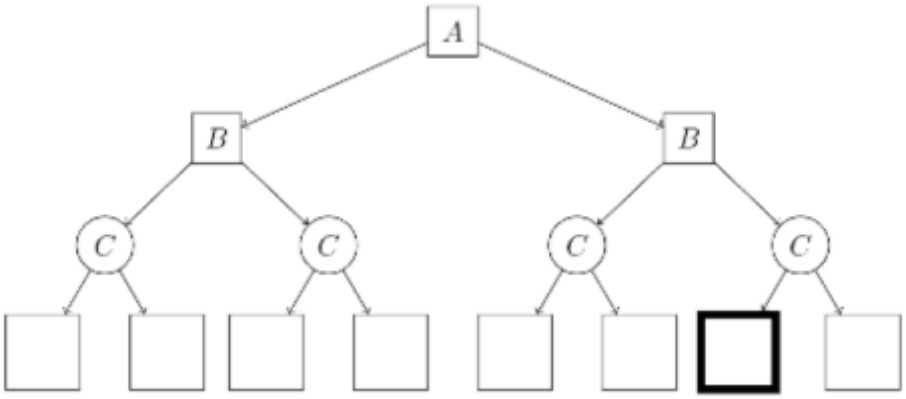
Các lá bị cắt: 6 (dưới c); 12 và -3 (dưới f); -2 và 10 (dưới h); -5 và -7 (dưới i). Tổng cộng **7 lá**.

Minimax phải đánh giá đủ **18 lá**. Số lá bị cắt: 1 (lá 6) + 2 (nút f) + 2 (nút h) + 2 (nút i) = 7. Alpha-Beta chỉ đánh giá 18 − 7 = **11 lá** (3, 0, -1, 1, 4, 10, -1, 0, -2, -4, -2), tiết kiệm 7/18 ≈ 39% mà kết quả vẫn bằng nhau.

Bài 2. Alpha-Beta trong cây có nút ngẫu nhiên

Đặt tên lá từ trái sang phải là L1 đến L8. A là MIN (gốc), B là MAX, C là chance với các con có xác suất bằng nhau, nên giá trị của C là trung bình cộng hai con. Quy ước không cắt khi bằng nhau, nên điều kiện cắt luôn là bất đẳng thức chặt.

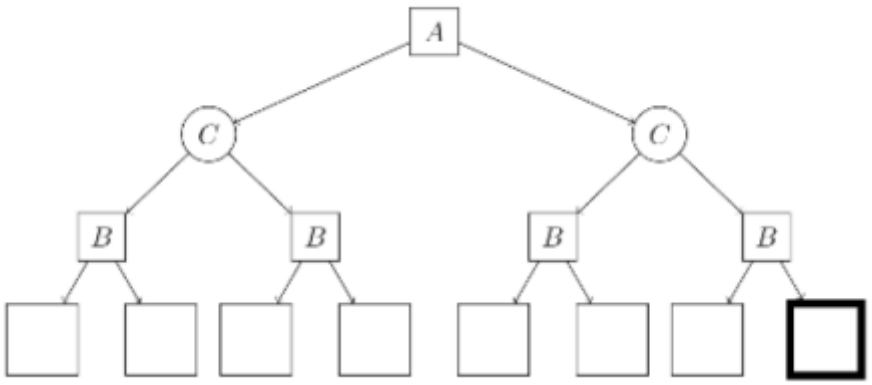
a) Trường hợp 1 (A → B → C → lá, lá in đậm là L7)



- Muốn bỏ qua L7 thì phải bỏ cả nút C thứ tư (L7, L8), tức là cắt nút B thứ hai ngay sau khi xét xong C3.
- Sau khi duyệt B thứ nhất: B1 = max((L1+L2)/2, (L3+L4)/2). Đây là β của A.
- Ở B thứ hai, sau khi tính C3 = (L5+L6)/2 thì α = C3. Điều kiện cắt: C3 > B1.
- Chọn số: L1 = L2 = L3 = L4 = 0, L5 = L6 = 4, L7 và L8 là số bất kỳ.
- Tính thử: C1 = (0+0)/2 = 0, C2 = (0+0)/2 = 0, B1 = max(0, 0) = 0 nên β = 0. C3 = (4+4)/2 = 4 nên α = 4. Vì 4 > 0 nên cắt L7 và L8.

→ Có thể

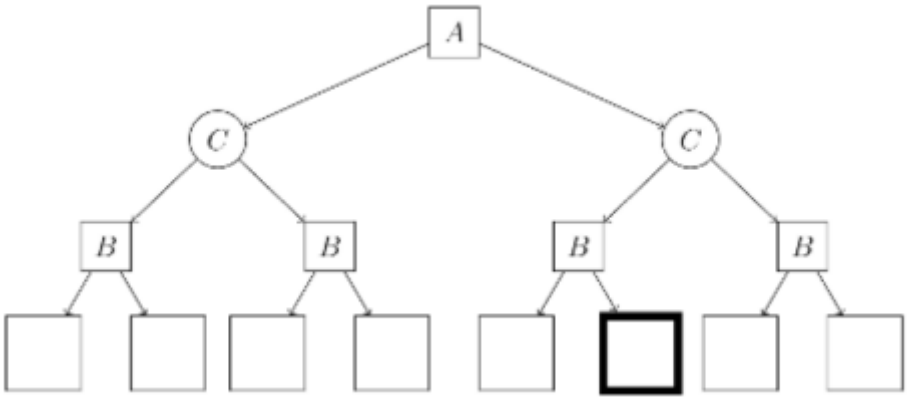
b) Trường hợp 2 (A → C → B → lá, lá in đậm là L8)



- Công thức: B1 = max(L1, L2), B2 = max(L3, L4), C1 = (B1+B2)/2; B3 = max(L5, L6), B4 = max(L7, L8), C2 = (B3+B4)/2; A = min(C1, C2).
- Sau khi biết C1 thì β = C1. Khi xét B4 và đã thấy L7 thì B4 ≥ L7, nên C2 ≥ (B3+L7)/2. Điều kiện cắt L8: (B3+L7)/2 > C1.
- Chọn số: L1 = L2 = L3 = L4 = 0, L5 = L6 = 4, L7 = 0, L8 là số bất kỳ.
- Tính thử: B1 = max(0, 0) = 0, B2 = 0, C1 = (0+0)/2 = 0 nên β = 0. B3 = max(4, 4) = 4. C2 ≥ (4+0)/2 = 2 > 0 = C1 nên cắt L8.

→ Có thể

c) Trường hợp 3 (A → C → B → lá, lá in đậm là L6)



- L6 nằm dưới B3 = max(L5, L6), và B3 chỉ cho biết B3 ≥ L5. Nút B4 chưa được duyệt.
- Để cắt L6 phải chứng minh C2 = (B3+B4)/2 > C1 ngay lúc đó. Nhưng B4 chưa biết và có thể nhỏ tùy ý (lá chỉ cần hữu hạn).
- Ví dụ: C1 = 0, L5 = 100. Nếu B4 = -1000 thì C2 ≥ (100 + (-1000))/2 = -450, có thể nhỏ hơn C1 = 0, khi đó A lại chọn C2 và L6 vẫn có thể ảnh hưởng kết quả.
- Không có cách gán số cho lá L1 đến L5 để chắc chắn C2 > C1, nên

→ Không thể

d) Vì sao cắt tỉa ở nút ngẫu nhiên khác Minimax

Ở Minimax, giá trị của nút MAX hay MIN chỉ phụ thuộc vào một con (con lớn nhất hoặc nhỏ nhất). Chỉ cần thấy một con đủ tốt hoặc đủ xấu là cho ra một cận (ví dụ MAX ≥ giá trị con đã xem), nên các con còn lại không thể thay đổi quyết định của nút cha và ta cắt được.

Nút ngẫu nhiên thì lấy trung bình có trọng số của tất cả các con. Một con chưa xem với giá trị rất lớn hoặc rất nhỏ vẫn làm thay đổi trung bình tùy ý, nên không thể kết luận gì từ vài con đã xem. Chỉ cắt được khi (1) đã xem hết các con của nút ngẫu nhiên (như trường hợp 1 và nút cuối ở trường hợp 2), hoặc (2) biết trước cận trên và cận dưới của giá trị lá thì mới tính được khoảng giá trị của trung bình. Vì vậy cắt tỉa trong cây expectimax kém hiệu quả hơn nhiều.

Bài 3. Câu hỏi đúng/sai

a) Luôn có $V_M \leq V_E$:

[x] Đúng [ ] Sai

Với mọi phân phối xác suất, kỳ vọng của một tập giá trị luôn ≥ giá trị nhỏ nhất của tập đó, nên tại mỗi nút MIN bị thay bằng nút ngẫu nhiên thì E[x] ≥ min(x). Chứng minh quy nạp từ lá lên: ở lá hai giá trị bằng nhau; nút MAX lấy max của các giá trị mà mỗi giá trị trong cây expectimax ≥ giá trị tương ứng trong cây minimax nên max cũng ≥;

nút MIN thay bằng kỳ vọng thì kỳ vọng ≥ min ≥ min của các giá trị minimax. Suy ra $V_E \geq V_M$.

b) Chính sách Minimax tối ưu bảo đảm lợi ích kỳ vọng ≥ V_M:

[x] Đúng [ ] Sai

Chính sách minimax bảo đảm ít nhất V_M ngay cả khi đối thủ chọn nước đi xấu nhất cho mình. Nếu đối thủ thay bằng ngẫu nhiên thì mọi kết quả đều không tệ hơn trường hợp xấu nhất, nên kỳ vọng ≥ V_M.

c) Chính sách Minimax tối ưu bảo đảm lợi ích kỳ vọng ≥ V_E:

[ ] Đúng [x] Sai

Minimax chỉ tối ưu khi đối thủ chơi hoàn hảo, nên có thể chọn sai khi đối thủ là ngẫu nhiên. Phản ví dụ: gốc MAX có hai nước đi. Nước 1 dẫn đến nút có hai lá (1, 1). Nước 2 dẫn đến nút có hai lá (0, 100) với xác suất 0.5 mỗi lá. Minimax: nước 1 cho 1, nước 2 cho min(0, 100) = 0, nên V_M = 1 và chính sách minimax chọn nước 1. Expectimax: nước 1 cho 1, nước 2 cho 0.5·0 + 0.5·100 = 50, nên V_E = 50. Chính sách minimax chỉ được kỳ vọng 1 < 50 = V_E.

Trò chơi một người, kết quả hành động ngẫu nhiên, thắng +1 và thua −1

[x] Minimax với tiện ích trạng thái cuối +1/−1

[x] Expectimax với tiện ích trạng thái cuối +1/−1

[x] Value Iteration với phần thưởng 0 ở trạng thái trung gian và +1/−1 ở kết quả thắng/thua

[ ] Không phương án nào

d) Giải thích và các giả định

- Minimax: dùng được nếu ta coi sự ngẫu nhiên của môi trường là một đối thủ xấu nhất (nút MIN). Cách này bảo thủ, chỉ cần biết các kết quả có thể xảy ra mà không cần biết xác suất. Nó bảo đảm mức tối thiểu nhưng có thể không tối đa hóa kỳ vọng.
- Expectimax: dùng được nếu biết phân phối xác suất của kết quả mỗi hành động và cây trò chơi hữu hạn. Nút ngẫu nhiên lấy kỳ vọng, với tiện ích ±1 thì giá trị bằng 2·P(thắng) − 1, nên tối đa hóa kỳ vọng cũng là tối đa hóa xác suất thắng.
- Value Iteration: dùng được nếu mô hình hóa thành MDP có mô hình chuyển trạng thái biết trước, phần thưởng 0 ở trạng thái trung gian và ±1 ở trạng thái

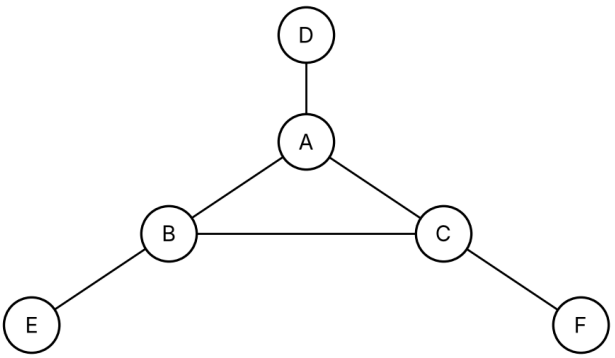
cuối. Với trò chơi hữu hạn (hoặc có hệ số chiết khấu $\gamma < 1$ khi có vòng lặp) thì hội tụ và cho chính sách giống expectimax.

Vì cả ba đều dùng được với giả định phù hợp nên không chọn "Không phương án nào".

Bài 4. CSP với đồ thị ràng buộc và lan truyền ràng buộc

a) Đồ thị ràng buộc

Mỗi sinh viên là một đỉnh, mỗi ràng buộc nhị phân là một cạnh. Danh sách kề: A: B, C, D; B: A, C, E; C: A, B, F; D: A; E: B; F: C.



Đồ thị ràng buộc với 6 cạnh: A-B, A-C, A-D, B-C, B-E, C-F.

b) Forward Checking sau khi gán A

Forward Checking chỉ loại giá trị ở các biến chưa gán có cạnh nối trực tiếp với biến vừa gán. Hàng xóm của A là B, C, D nên miền của B, C, D có thể thay đổi. E và F không kề A nên không đổi.

c) Gán tiếp B (đã Forward Checking ở A)

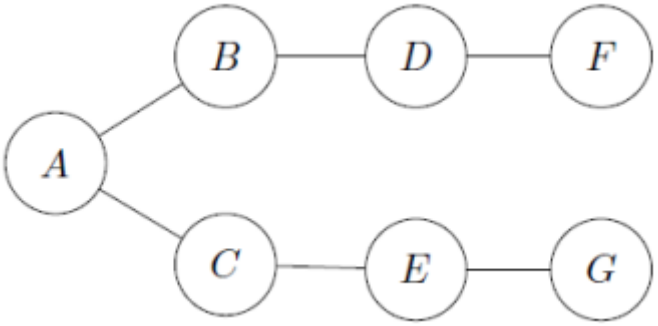
Hàng xóm của B là A, C, E. A đã gán, nên miền của C và E có thể thay đổi. D không kề B nên không bị ảnh hưởng.

d) Duy trì Arc Consistency sau khi gán A

- Gán A: miền B, C, D (hàng xóm của A) bị thu hẹp.
- Miền B đổi nên xét lại các cung tới B từ hàng xóm của B: C và E (A đã gán). Vậy C, E có thể đổi.
- Miền C đổi nên xét lại các cung từ hàng xóm của C: B và F. Vậy F có thể đổi.
- E chỉ kề B, F chỉ kề C, D chỉ kề A, nên không lan thêm.
- Kết quả: miền của **B, C, D, E, F** có thể thay đổi.

e) Gán tiếp B và duy trì Arc Consistency

- Gán B: miền B chỉ còn một giá trị.
- Hàng xóm của B là A (đã gán), C, E nên C và E có thể đổi.
- Miền C đổi: hàng xóm của C là A, B (đã gán) và F nên F có thể đổi.
- E chỉ kề B nên dừng. D chỉ kề A mà miền A không đổi nên D không bị ảnh hưởng.
- Kết quả: miền của **C, E, F** có thể thay đổi, D thì không.



Thuật toán mới: AC ban đầu và sau mỗi lần đã gán một số chẵn biến

Đồ thị ở Hình 3 có các cạnh A-B, B-D, D-F, A-C, C-E, E-G, là một cây, thực ra là đường đi F - D - B - A - C - E - G (7 đỉnh). Các giả định mình dùng:

- Bài toán có nghiệm, miền các biến có d giá trị và AC ban đầu không làm miền rỗng.
- Giữa các lần chạy AC chỉ kiểm tra ràng buộc với các biến đã gán (backtracking thường). AC chạy ở đầu và sau khi đã gán 2, 4, 6 biến.
- Một lần backtrack là một lần phải hủy phép gán vì gặp ngõ cụt (không có giá trị hợp lệ hoặc AC làm miền rỗng) và thử giá trị khác.
- Trường hợp xấu nhất: các miền còn đủ d giá trị và giá trị đúng luôn được thử cuối cùng.

Tính chất dùng: với cây đã arc-consistent và các miền khác rỗng thì mỗi giá trị trong mỗi miền đều mở rộng được thành một nghiệm. Do đó nếu các biến đã gán tạo thành một đoạn liên tục trên đường đi (liên thông) và nhất quán với nhau thì luôn mở rộng được, không backtrack. Chỉ khi gán hai phần tách rời nhau (ở giữa còn biến chưa gán) thì hai giá trị chọn độc lập mới có thể mâu thuẫn.

Cách tính chung: AC chạy sau lần gán thứ 2, 4, 6 nên ta chia thành các cặp gán (1,2), (3,4), (5,6) và lần 7. Trong một cặp, nếu giữa hai biến mới còn biến chưa gán thì xấu nhất mất $d - 1$ backtrack, còn lại thì 0. Bảng bên dưới tính từng cặp.

f) Thứ tự A → B → C → D → E → F → G

Các biến đã gán luôn là một đoạn liên tục: {A}, {B,A}, {B,A,C}, {D,B,A,C}, ... mỗi biến mới kề với một đầu của đoạn. Biến mới luôn có giá trị tương thích với biến kề đã gán (nhờ AC), và giá trị đó còn mở rộng tiếp được. **Số backtrack xấu nhất = 0.**

Cặp gán	Hai biến	Có biến chưa gán nằm giữa?	Backtrack xấu nhất
Lần 1, 2	A, B	Không (A và B kề nhau)	0
Lần 3, 4	C, D	Không (nối với nhau qua A, B đã gán)	0
Lần 5, 6	E, F	Không (nối với nhau qua C, A, B, D đã gán)	0
Lần 7	G	Không có	0
Tổng			0

g) Thứ tự G → E → C → A → B → D → F

Đây là đi dọc theo đường từ một đầu sang đầu kia, mỗi biến kề với biến vừa gán, các biến đã gán luôn liên tục. **Số backtrack xấu nhất = 0.**

Cặp gán	Hai biến	Có biến chưa gán nằm giữa?	Backtrack xấu nhất
Lần 1, 2	G, E	Không (G và E kề nhau)	0
Lần 3, 4	C, A	Không (C và A kề nhau)	0
Lần 5, 6	B, D	Không (B và D kề nhau)	0
Lần 7	F	Không có	0
Tổng			0

h) Thứ tự G → F → E → D → C → B → A

Thứ tự này gán các biến ở hai đầu đường đi trước nên các phần đã gán bị tách rời:

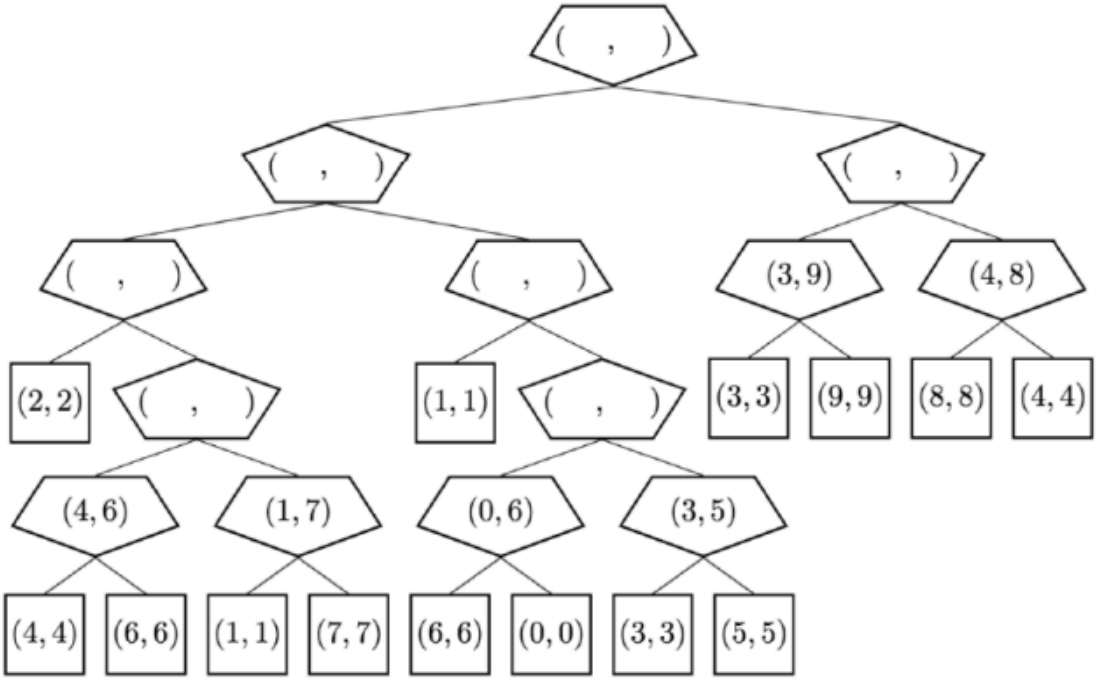
- Gán G rồi F (không kề nhau, lúc này cách nhau cả đường F-D-B-A-C-E-G). Sau lần gán thứ 2 chạy AC, nếu f mâu thuẫn với g thì miền rỗng. Với g cố định luôn có ít nhất một f hợp lệ (do g có trong một nghiệm), xấu nhất thử sai d − 1 giá trị: **d − 1 backtrack**.
- Gán E (kề G), không sinh lỗi vì {E,G} liên tục và {F} đã nhất quán nhờ AC ở bước 2.
- Gán D (kề F nhưng cách E bởi B, A, C). Sau lần gán thứ 4 chạy AC để phát hiện D và E mâu thuẫn, xấu nhất d − 1 lần thử sai: **d − 1 backtrack**.
- Gán C (kề E) thì không lỗi. Gán B (kề D) rồi chạy AC sau lần gán thứ 6: B và C bị A ngăn cách nên có thể mâu thuẫn, xấu nhất **d − 1 backtrack**.

- Gán A: AC ở bước 6 đã bảo đảm miền A khác rỗng và tương thích cả B lẫn C, nên không backtrack.

Số backtrack xấu nhất = 3(d − 1). Kết luận: thứ tự gán dọc theo cấu trúc cây (f, g) không cần backtrack, còn thứ tự ngược nhảy cách quãng (h) tốn thêm.

Cặp gán	Hai biến	Có biến chưa gán nằm giữa?	Backtrack xấu nhất
Lần 1, 2	G, F	Có (E, C, A, B, D)	d − 1
Lần 3, 4	E, D	Có (C, A, B)	d − 1
Lần 5, 6	C, B	Có (A)	d − 1
Lần 7	A	Không có	0
Tổng			3(d − 1)

Bài 5. Cây trò chơi Pacman và Casper



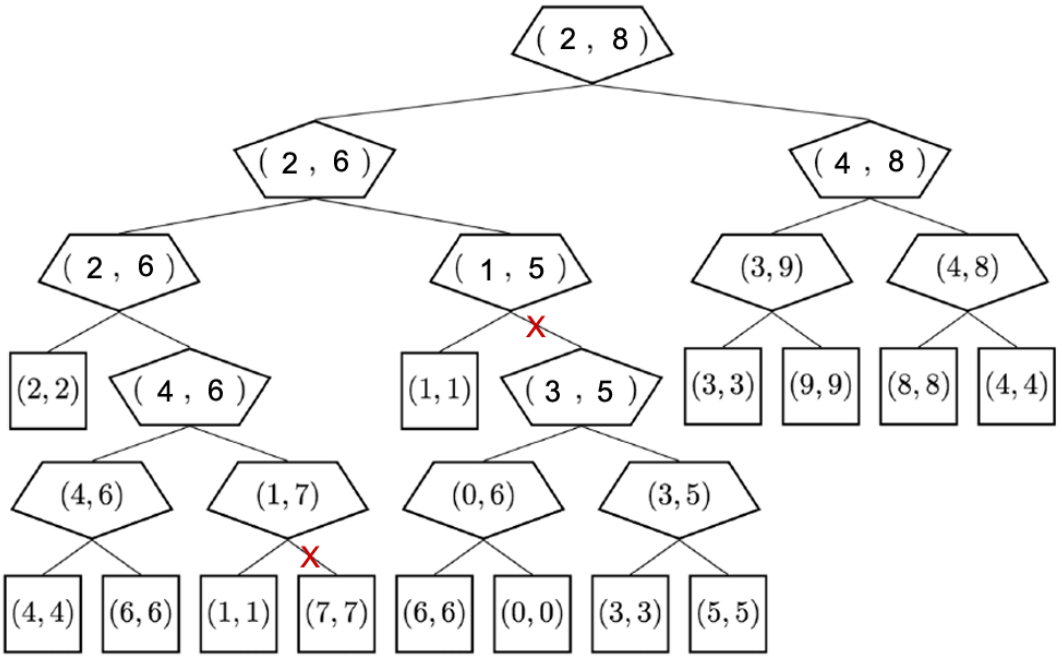
Ta nhận ra loại nút nhờ cặp giá trị ở nút Casper có sẵn trong hình: (3, 9) có hai lá (3,3), (9,9) nên u = min = 3, v = max = 9. Vậy ở nút Casper, u = min các u của con (Pacman nghĩ Casper muốn hại mình), còn v = max các v của con (thực tế Casper muốn giúp Pacman). Ở nút Pacman, u = max các u của con, và Pacman chọn con có u lớn nhất, nên v của nút đó là v của đúng con được chọn (không phải max v).

a) Điền các cặp (u, v)

Tính từ dưới lên (gọi P là nút Pacman, C là nút Casper):

- Casper tầng dưới cùng: (4,6) từ lá (4,4),(6,6); (1,7) từ (1,1),(7,7); (0,6) từ (6,6),(0,0); (3,5) từ (3,3),(5,5), tất cả đã có sẵn trong hình.
- Nút Pacman có hai con (4,6) và (1,7) (nhánh trái): u = max(4, 1) = 4, chọn con (4,6) nên v = 6. Được (4, 6).
- Nút Pacman có hai con (0,6) và (3,5) (nhánh giữa): u = max(0, 3) = 3, chọn (3,5) nên v = 5. Được (3, 5).
- Casper có con là lá (2,2) và nút (4,6): u = min(2, 4) = 2; v = max(2, 6) = 6. Được (2, 6).
- Casper có con là lá (1,1) và nút (3,5): u = min(1, 3) = 1; v = max(1, 5) = 5. Được (1, 5).
- Pacman tầng 1 bên trái: con là (2,6) và (1,5). u = max(2, 1) = 2, chọn (2,6) nên v = 6. Được (2, 6).
- Pacman tầng 1 bên phải: con là (3,9) và (4,8). u = max(3, 4) = 4, chọn (4,8) nên v = 8. Được (4, 8).
- Gốc (Casper): u = min(2, 4) = 2; v = max(6, 8) = 8. Được (2, 8).

Khác biệt giữa u và v ở nút Casper: u là giá trị theo niềm tin sai của Pacman (Casper là kẻ thù, chọn min). v là giá trị theo hành vi thực của Casper (Casper thân thiện, chọn max cho Pacman). Ví dụ ở (3,9), Pacman tưởng Casper sẽ chọn 3 nhưng thực tế Casper chọn 9. Pacman vẫn ra quyết định theo u, nên giá trị thực tế của cả cây là 8 chứ không phải 9 (Pacman bị lệch do hiểu nhầm).



Hình 4 (đã điền). Cặp (u, v) ở mọi nút; các nhánh X có thể không cần duyệt.

b) Các nhánh có thể không cần duyệt

Casper cần biết giá trị thực v của cả cây, nên phải mô phỏng đúng lựa chọn của Pacman (theo u). Vì vậy chỉ được bỏ qua phần nào không thể thay đổi lựa chọn của Pacman. Duyệt trái sang phải:

- Lá (7,7) dưới nút (1,7): Pacman đã có con (4,6) với u = 4. Nút (1,7) sau khi xem lá (1,1) thì $u \leq 1 < 4$, nên Pacman chắc chắn không chọn nó, không cần xem lá (7,7).
- Toàn bộ nhánh bên phải của nút Casper (1,5), tức nút Pacman (3,5) cùng hai nút (0,6), (3,5) và bốn lá bên dưới: Pacman ở tầng 1 đã có con (2,6) với u = 2. Nút Casper này sau khi xem lá (1,1) thì $u \leq 1 < 2$, Pacman sẽ không chọn nó, nên phần còn lại không ảnh hưởng đến v của cả cây.

Nhánh bên phải của gốc không cắt được: muốn biết v ở gốc (là max của hai nhánh) thì phải biết đủ giá trị thực của nhánh phải, và ở nút Pacman bên phải cũng phải xem đủ hai lá của mỗi nút Casper vì chưa có bất đẳng thức nào để loại.

c) Nếu Pacman biết Casper thân thiện

Khi đó cả hai cùng muốn tăng tiện ích của Pacman, nên mọi nút đều là nút MAX và giá trị của cây bằng lá lớn nhất, đó là 9 (đường gốc → nhánh phải → nút (3,9) → lá (9,9)). Khi biết đúng thì niềm tin (u) và thực tế (v) trùng nhau và cùng bằng 9. So với trước đó, niềm tin sai cho giá trị 2 còn giá trị thực tế chỉ đạt 8, vì Pacman chọn nhánh (4,8) thay vì nhánh có lá 9.

Bài 6. Thuật toán tìm kiếm: tính đầy đủ và tối ưu

Thuật toán	Điều kiện đầy đủ	Điều kiện tối ưu
BFS	Hệ số nhánh b hữu hạn (và có nghiệm ở độ sâu hữu hạn). Đúng cho cả tìm kiếm trên cây và trên đồ thị.	Chi phí các bước bằng nhau (hoặc chi phí không giảm theo độ sâu), khi đó lời giải nông nhất cũng rẻ nhất. Chi phí khác nhau thì không tối ưu.
DFS	Trên cây: không đầy đủ nếu có nhánh vô hạn hoặc vòng lặp. Trên đồ thị (có tập đã thăm): đầy đủ khi không gian trạng thái hữu hạn.	Không tối ưu: trả về lời giải đầu tiên gặp trên nhánh đi sâu, chưa chắc nông nhất hay rẻ nhất.
IDS	Hệ số nhánh b hữu hạn và có nghiệm ở độ sâu hữu hạn.	Chi phí các bước bằng nhau (giống BFS). Chi phí khác nhau thì không tối ưu.
A*	b hữu hạn và chi phí mỗi cạnh $\geq \varepsilon > 0$ (để f tăng vượt mọi cận hữu hạn).	Tìm kiếm trên cây: heuristic chấp nhận được (admissible: $h(n) \leq h^*(n)$). Tìm kiếm trên đồ thị:

Thuật toán	Điều kiện đầy đủ	Điều kiện tối ưu
		heuristic nhất quán (consistent: $h(n) \leq c(n, n') + h(n')$); chỉ admissible thì chưa đủ nếu không mở lại nút đã đóng.

Vì sao chi phí bước và heuristic ảnh hưởng đến kết luận

- BFS và IDS chỉ tối thiểu số bước, không phải tổng chi phí. Nếu các cạnh có chi phí khác nhau thì đường ít bước nhất chưa chắc rẻ nhất, nên mất tính tối ưu (cần UCS hoặc A*).
- Nếu có cạnh chi phí bằng 0 hoặc âm (hoặc nhỏ dần về 0) thì A* và UCS có thể lặp vô hạn hoặc không dừng, mất tính đầy đủ, nên cần chi phí $\geq \varepsilon > 0$.
- Heuristic phải không đánh giá quá cao. Nếu $h(n) > h^*(n)$ thì A* có thể loại nhầm nút nằm trên đường tối ưu. Khi tìm kiếm trên đồ thị, nếu h không nhất quán thì một nút có thể được mở rộng lần đầu qua đường đắt hơn rồi bị đóng, và đường rẻ hơn tìm thấy sau đó bị bỏ qua.
- Heuristic $h = 0$ thì A* trở thành UCS, còn heuristic càng sát h^* (vẫn admissible) thì càng ít nút bị mở rộng.

Bài 7. A* khi chi phí một cạnh thay đổi

Gọi C^* là chi phí tối ưu cũ. Ý chính: một đường đi chứa cạnh $X \rightarrow Y$ thì chi phí đổi đúng n (một đường đơn giản dùng cạnh đó một lần), còn đường không chứa cạnh đó giữ nguyên.

a) Tăng n, cạnh thuộc đường tối ưu cũ và đã được khám phá

Chi phí của đường cũ tăng lên $C^* + n$, còn các đường khác không chứa cạnh này giữ nguyên ($\geq C^*$). Nên chi phí tối ưu mới nằm trong $[C^*, C^* + n]$, và đường tối ưu có thể đổi sang một đường khác từng bị xem là kém hơn. Cách cập nhật mà không chạy lại từ đầu: tăng g thêm n cho Y và các nút con cháu trong cây tìm kiếm đi qua cạnh $X \rightarrow Y$, đưa chúng về lại frontier với f mới, giữ nguyên các nút khác đã khám phá, rồi tiếp tục A* cho đến khi nút đích được lấy ra với f nhỏ nhất. Heuristic vẫn hợp lệ (vì chi phí cạnh chỉ tăng nên $h(X) \leq c + n + h(Y)$ vẫn đúng). Nếu mọi nút frontier có $f \geq C^* + n$ thì đường cũ vẫn tối ưu với chi phí $C^* + n$.

b) Giảm n, cạnh thuộc đường tối ưu cũ và đã được khám phá

Đường cũ giảm còn $C^* - n$. Các đường khác hoặc không chứa cạnh (vẫn $\geq C^* > C^* - n$), hoặc chứa cạnh (trước đó $\geq C^*$, giờ giảm n nên $\geq C^* - n$). Do đó đường cũ vẫn là

tối ưu, chi phí mới là C* − n, chỉ cần trừ n vào chi phí lưu của đường, không cần chạy lại.

c) Tăng n, cạnh không thuộc đường tối ưu cũ nhưng đã được khám phá
Đường tối ưu cũ không chứa cạnh này nên chi phí không đổi, còn các đường khác chỉ đắt thêm hoặc giữ nguyên. Vậy đường tối ưu và chi phí tối ưu đều không đổi, không cần cập nhật gì.

d) Giảm n, cạnh không thuộc đường tối ưu cũ nhưng đã được khám phá
Có thể có đường mới tốt hơn đi qua X → Y. Mọi đường dùng cạnh này giảm đúng n và trước đó ≥ C*, nên chi phí tối ưu mới nằm trong [C* − n, C*]. Cách làm: tính g'(Y) = g(X) + (c − n). Nếu g'(Y) < g(Y) thì cập nhật g(Y), đưa Y trở lại frontier với f mới và lan truyền mức giảm xuống các nút con cháu của Y (mở lại các nút đó nếu cần), rồi tiếp tục A* cho đến khi nút đích được lấy ra. Nếu g'(Y) ≥ g(Y) thì không cải thiện được gì và đường cũ vẫn tối ưu.

e) Tăng n, cạnh không thuộc đường tối ưu cũ và chưa được khám phá
Cạnh chưa được dùng và cũng không nằm trên đường tối ưu, chi phí tăng chỉ làm các đường chứa nó đắt thêm. Vậy chi phí tối ưu và đường tối ưu không đổi. Khi cạnh được khám phá sau này thì cứ dùng chi phí mới c + n.

f) Giảm n, cạnh không thuộc đường tối ưu cũ và chưa được khám phá
Vì X chưa được mở rộng nên f(X) = g(X) + h(X) ≥ C* khi A* dừng. Đường qua X → Y nay rẻ hơn n, nên chi phí tối ưu mới nằm trong [C* − n, C*] và có thể thấp hơn. Cách làm: cập nhật heuristic của X để vẫn hợp lệ, h(X) := min(h(X), (c − n) + h(Y)), tính lại f của X, rồi tiếp tục A* từ frontier hiện tại. Nếu sau khi mở rộng không có đường nào qua X → Y rẻ hơn C* thì đường cũ vẫn là tối ưu.

Tóm lại: tăng chi phí ngoài đường tối ưu (c, e) không đổi gì, giảm chi phí trên đường tối ưu (b) thì đường giữ nguyên, các trường hợp còn lại (a, d, f) cần cập nhật g hoặc f rồi chạy tiếp A* từ frontier thay vì làm lại từ đầu.

Câu	Tình huống	Chi phí tối ưu mới	Cách làm
a	Tăng n, trên đường tối ưu, đã khám phá	Nằm trong [C*, C* + n]	Tăng g thêm n cho Y và con cháu, đưa về frontier, chạy tiếp A*
b	Giảm n, trên đường tối ưu, đã khám phá	C* − n, đường giữ nguyên	Chỉ cần trừ n vào chi phí đường
c	Tăng n, ngoài đường tối ưu, đã khám phá	Không đổi (C*)	Không phải làm gì

Câu	Tình huống	Chi phí tối ưu mới	Cách làm
d	Giảm n, ngoài đường tối ưu, đã khám phá	Nằm trong [C* − n, C*]	Tính g mới của Y, nếu nhỏ hơn thì cập nhật, lan xuống con cháu và chạy tiếp A*
e	Tăng n, ngoài đường tối ưu, chưa khám phá	Không đổi (C*)	Không phải làm gì, dùng chi phí mới khi khám phá
f	Giảm n, ngoài đường tối ưu, chưa khám phá	Nằm trong [C* − n, C*]	Cập nhật h(X) và f(X), chạy tiếp A* từ frontier

Bài 8. Tác tử thông minh và môi trường

Tác tử A không dùng percept (f_A()), B chỉ dùng percept hiện tại (f_B(percept)), C dùng toàn bộ lịch sử percept (f_C(percepts)). Tác tử C có thể bỏ qua lịch sử để làm giống B, B có thể bỏ qua percept để làm giống A, nên khả năng của C ≥ B ≥ A.

a) Quan sát đầy đủ, xác định, rời rạc, một tác tử, tĩnh

Các lớp tác tử: B và C. Quan sát đầy đủ nên percept chính là trạng thái, và hành động tối ưu chỉ phụ thuộc trạng thái hiện tại (môi trường xác định và tĩnh), vì vậy một hàm f(percept) là đủ. A thì không được vì không nhìn thấy trạng thái, mà hành động đúng phụ thuộc trạng thái.

b) Quan sát một phần, xác định, rời rạc, một tác tử, tĩnh

Chỉ có lớp C. Percept hiện tại không cho biết đủ trạng thái, hành động tốt có thể phụ thuộc vào những gì đã quan sát trước đó (ví dụ tác tử phải nhớ đã xem chỗ nào, hay suy ra trạng thái từ chuỗi percept). Môi trường xác định và tĩnh nên từ lịch sử percept tác tử suy ra được tập trạng thái có thể (belief state) và hành động hợp lý. B chỉ thấy percept hiện tại nên không đủ, A càng không.

c) Quan sát một phần, ngẫu nhiên, rời rạc, một tác tử, tĩnh

Chỉ có lớp C. Cũng như câu b, cần lịch sử percept để duy trì belief state, nay là phân phối xác suất trên các trạng thái. Chính sách tối ưu có thể chọn tất định theo lịch sử, nên tác tử C vẫn dựng lại được belief từ percepts (các hành động cũ được tính lại từ f_C). B và A không đủ thông tin. Tính ngẫu nhiên của môi trường không giúp B hay A thoát khỏi việc cần nhớ.

d) Có tồn tại một môi trường mà mọi tác tử đều hành động hợp lý?

Đúng. Tác tử hợp lý là tác tử tối đa hóa kỳ vọng thước đo hiệu quả. Nếu môi trường mà mọi hành động đều cho cùng một kết quả (ví dụ chỉ có một hành động duy nhất, hoặc thước đo hiệu quả hoàn toàn không phụ thuộc hành động) thì bất kỳ tác tử nào, kể cả tác tử chọn ngẫu nhiên, cũng đạt giá trị tối đa nên đều hợp lý.